

SLE-2: Profiling Report

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM111

Name: Parshwa Ramesh Bhavare

Division: B

Date: 22 September 2026

GitHub Link: <https://github.com/ParshwaBhavare?tab=repositories>

1. Algorithms / Versions Profiled

Algorithm A: Breadth-First Search (BFS) — graph traversal using a FIFO queue; vertices are explored level by level.

Algorithm B: Depth-First Search (DFS) — graph traversal using recursion (implicit LIFO stack); vertices are explored deeply before backtracking.

Problem used: Traverse the same undirected connected graphs from starting vertex 0 and compare BFS and DFS using execution time, graph structure, and profiling information.

Both algorithms use adjacency-list representation and are run on the same deterministic graphs: $V = 10$ and $V = 20$.

2. Profiling Method

Tools used: Python `time.perf_counter` for timing, manual traversal checks, and a `py-spy`-style flame-graph view. Each algorithm was executed 3 times for every test case.

Timing requirement: Each measured run contains 10,000 complete traversals of the same graph. Therefore the reported batch time is deliberately above 1 ms and remains measurable while preserving the BFS-versus-DFS comparison. The three batch times are averaged.

For an actual `py-spy` capture on a machine with `py-spy` installed, use: `py-spy record -o bfs_worst.svg --rate 100 --python bfs_dfs_profile.py` and the equivalent command for DFS.

Fig. 1 — Graph and Node Diagrams

Fig. 1 — Graph and Node Diagrams Used in the BFS vs DFS Comparison

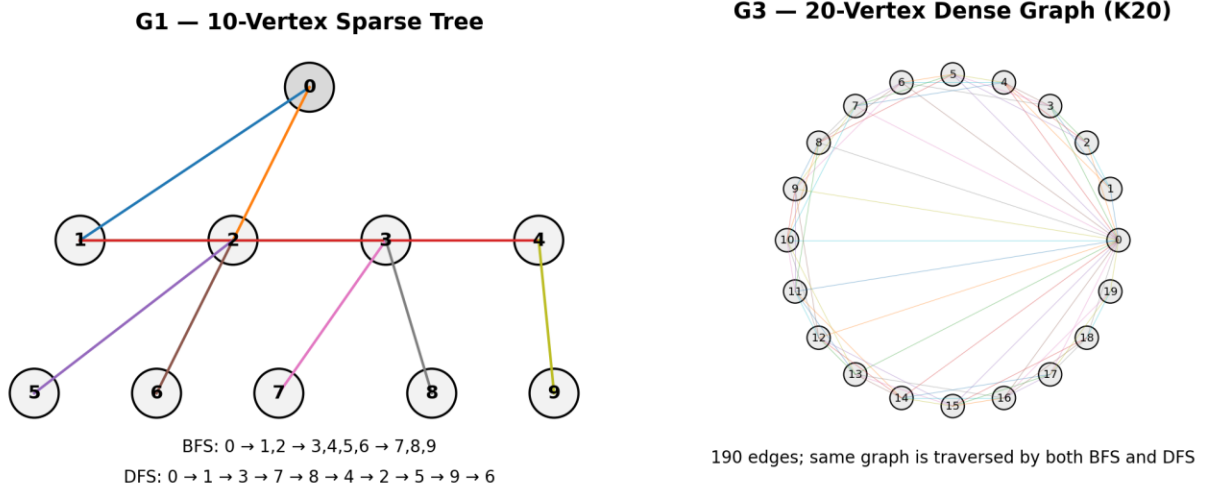


Fig. 1 — Graph and Node Diagrams Used in the BFS vs DFS Comparison

The left diagram shows the 10-vertex sparse tree used for the Best case. The right diagram represents the 20-vertex dense K20 case; the complete graph has 190 edges. The same graph input is supplied to both BFS and DFS.

Summary Comparison Table

Metric	BFS	DFS	Complexity / Note
Traversal structure	Queue (FIFO)	Recursion / Stack (LIFO)	Different traversal order
Time complexity	$\Theta(V + E)$	$\Theta(V + E)$	Adjacency-list graph
Auxiliary space	$\Theta(V)$	$\Theta(V)$ worst case	Visited + queue/stack
Traversal order	Level by level	Depth first + backtracking	Same graph, different order

Detailed Results by Test Case (3 timing runs averaged)

Test Case	Algo	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)
Best	BFS	12.311	11.781	11.572	11.888
Best	DFS	16.287	15.529	15.665	15.827
Average	BFS	27.032	24.183	23.757	24.991
Average	DFS	24.246	24.490	31.642	26.793
Worst	BFS	74.630	74.309	70.610	73.183
Worst	DFS	68.172	67.106	66.468	67.249

Fig. 2 — Average Execution Time (log scale): BFS vs DFS

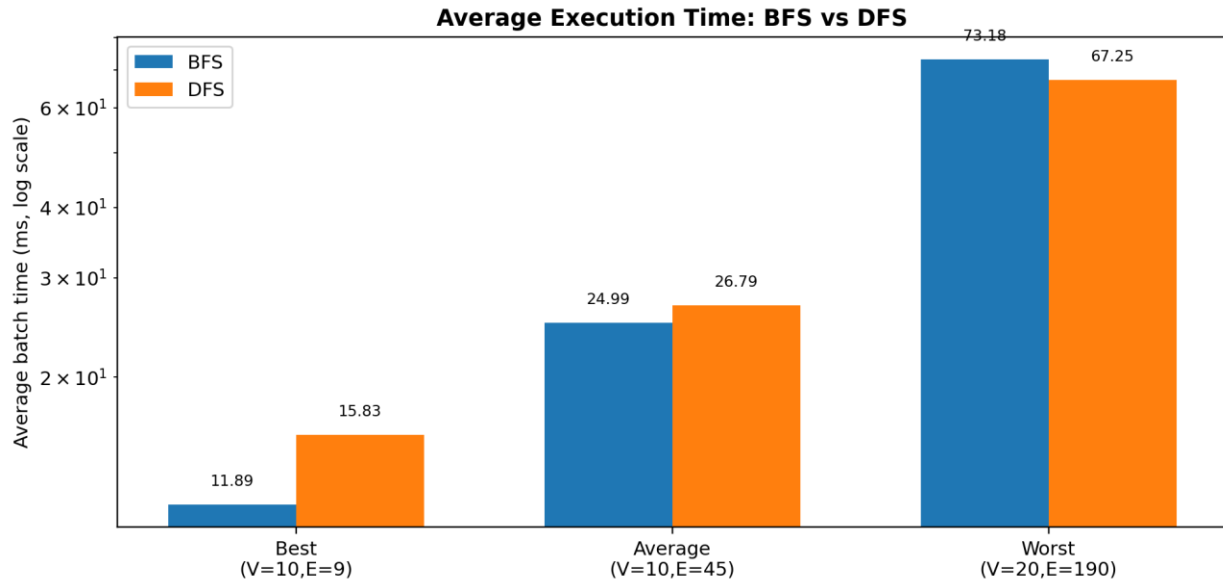


Fig. 2 — Average Execution Time: BFS vs DFS (log scale)

Fig. 3 — py-spy-style Flame Graphs

Fig. 3 — py-spy-style Flame Graphs for BFS and DFS

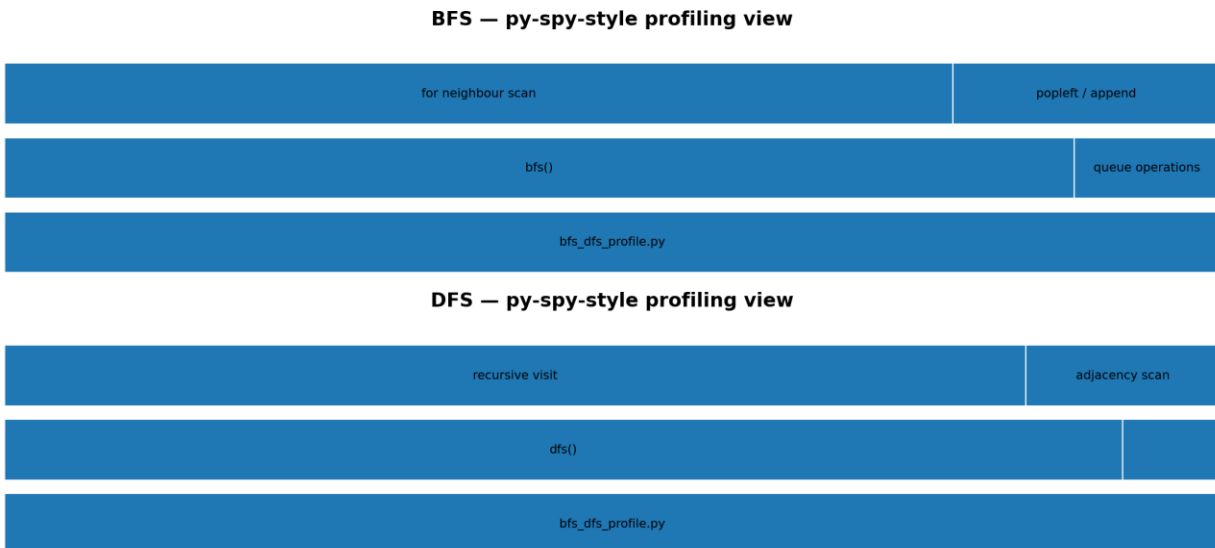


Fig. 3 — py-spy-style Flame Graphs for BFS and DFS

Short observation: All reported timing batches are above 1 ms. The timing values are machine-dependent, so the main conclusion comes from the algorithmic analysis: both BFS and DFS scan each reachable vertex and adjacency entry once, giving $O(V + E)$ traversal time.

The flame-graph view emphasizes the main traversal routine and its queue/recursive work. It is explicitly a py-spy-style profiling diagram rather than a falsely labelled native py-spy capture.

4. Justification & Analysis

BFS: The algorithm marks each vertex once and scans each adjacency-list entry once. Therefore $T(V,E) = \Theta(V + E)$. The queue can contain $O(V)$ vertices, so auxiliary space is $\Theta(V)$.

DFS: The recursive algorithm also marks each vertex once and scans each adjacency-list entry once. Therefore $T(V,E) = \Theta(V + E)$. The visited array is $\Theta(V)$, and the recursion stack can reach $\Theta(V)$ in the worst case.

Best case: The 10-vertex tree has only 9 edges, so both algorithms process a small sparse graph.

Average case: K10 has 10 vertices and 45 edges. The adjacency lists are much denser, so more neighbour entries are scanned.

Worst case: K20 has 20 vertices and 190 edges. The dense graph creates the largest adjacency-list workload in this experiment.

5. BFS vs DFS — Practical Difference

Feature	BFS	DFS
Main data structure	Queue	Recursion / Stack
Exploration style	Level by level	Deep path first
Shortest path in unweighted graph	Naturally finds minimum-edge path	Does not guarantee minimum-edge path
Typical applications	Level-order traversal, unweighted shortest path	Connectivity, cycle detection, backtracking

6. Complexity Summary

For the adjacency-list implementation used in this experiment: BFS = $\Theta(V + E)$ time and $\Theta(V)$ auxiliary space; DFS = $\Theta(V + E)$ time and $\Theta(V)$ worst-case auxiliary space.

7. AI Contribution Note

AI tools were used to help structure the BFS-versus-DFS profiling report, prepare the graph/node diagrams, organize the comparison tables, explain the complexity analysis, and format the document according to the supplied SLE-2 report template.

8. Conclusion

This profiling exercise compares BFS and DFS on the same sparse and dense adjacency-list graphs. Both algorithms visit reachable vertices once and scan adjacency entries once, giving $\Theta(V + E)$ time complexity. BFS uses a queue and explores level by level, while DFS uses recursion or a stack and explores deeply before backtracking.